# Mesh GNNs (MeshGraphNet) for vortex shedding

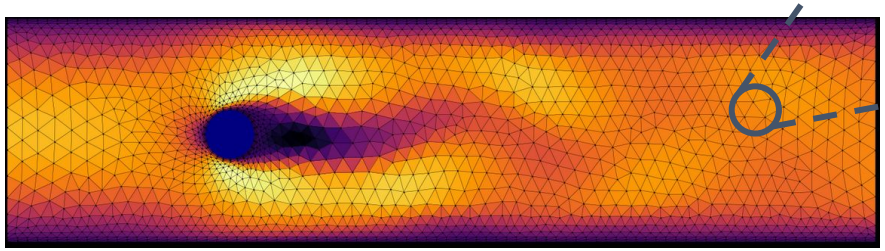
Published as a conference paper at ICLR 2021

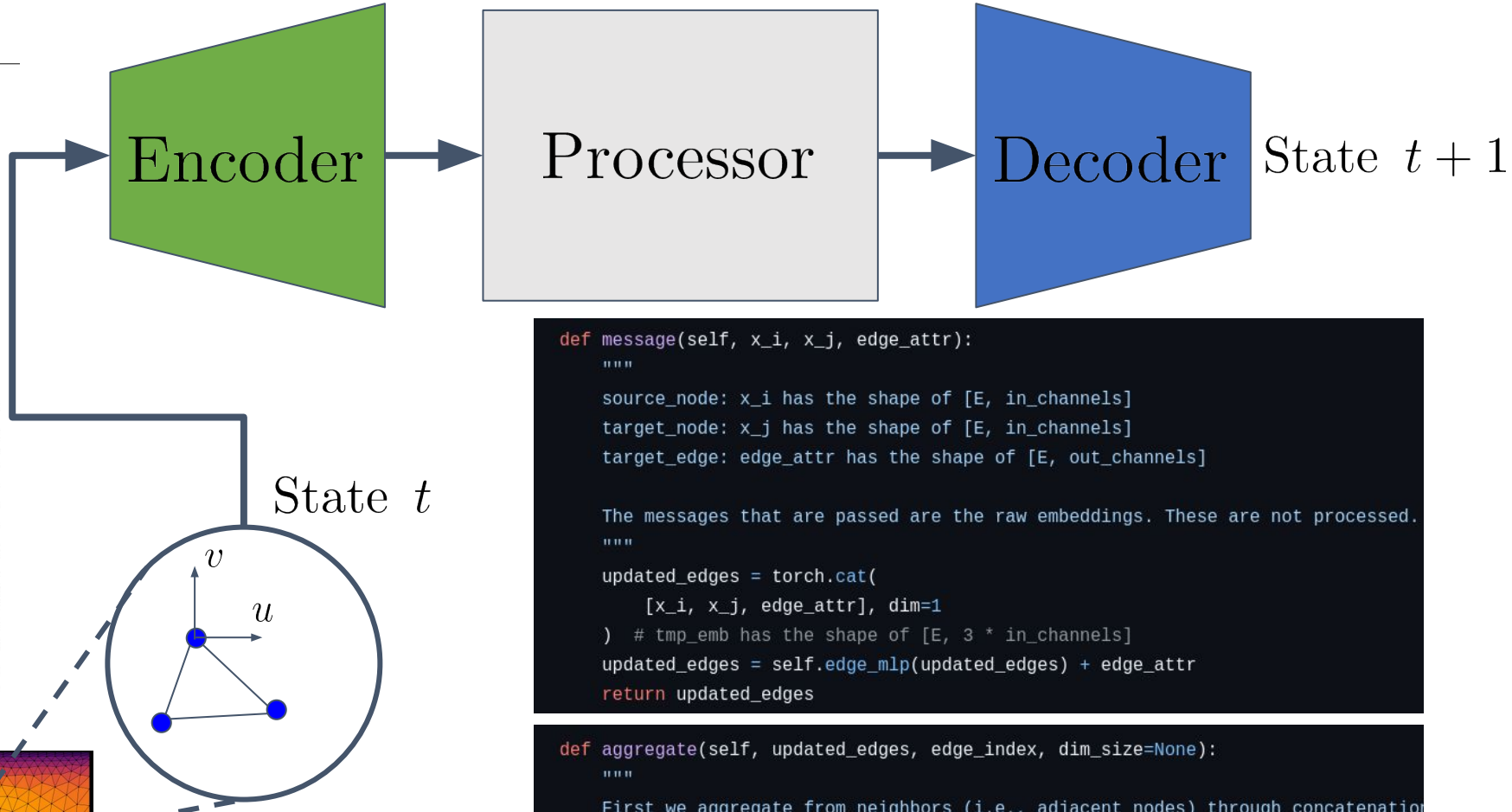
## Learning Mesh-Based Simulation with Graph Networks

**Tobias Pfaff,* Meire Fortunato,* Alvaro Sanchez-Gonzalez,* Peter W. Battaglia**
Deepmind, London, UK
{tpfaff,meirefortunato,alvarosg,peterbattaglia}@google.com

### Abstract

Mesh-based simulations are central to modeling complex physical systems in many disciplines across science and engineering. Mesh representations support powerful numerical integration methods and their resolution can be adapted to strike favorable trade-offs between accuracy and efficiency. However, high-dimensional scientific simulations are very expensive to run, and solvers and parameters must often be tuned individually to each system studied. Here we introduce MeshGraphNets, a framework for learning mesh-based simulations using graph neural networks. Our model can be trained to pass messages on a mesh graph and to adapt the mesh discretization during forward simulation. Our results show it can accurately predict the dynamics of a wide range of physical systems, including aerodynamics, structural mechanics, and cloth. The model's adaptivity supports learning resolution-independent dynamics and can scale to more complex state spaces at test time. Our method is also highly efficient, running 1-2 orders of magnitude faster than the simulation on which it is trained. Our approach broadens the range of problems on which neural network simulators can operate and promises to improve the efficiency of complex, scientific modeling tasks.






```python
def message(self, x_i, x_j, edge_attr):
    """
    source_node: x_i has the shape of [E, in_channels]
    target_node: x_j has the shape of [E, in_channels]
    target_edge: edge_attr has the shape of [E, out_channels]

    The messages that are passed are the raw embeddings. These are not processed.
    """
    updated_edges = torch.cat(
        [x_i, x_j, edge_attr], dim=1
    )  # tmp_emb has the shape of [E, 3 * in_channels]
    updated_edges = self.edge_mlp(updated_edges) + edge_attr
    return updated_edges
```

```python
def aggregate(self, updated_edges, edge_index, dim_size=None):
    """
    First we aggregate from neighbors (i.e., adjacent nodes) through concatenatio
    then we aggregate self message (from the edge itself). This is streamlined
    into one operation here.
    """
    # The axis along which to index number of nodes.
    node_dim = 0
    out = torch_scatter.scatter(
        updated_edges, edge_index[0, :], dim=node_dim, reduce="sum"
    )
    return out, updated_edges
```